SC4064 GPU Programming

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Introduction: GPU Architecture, SIMT and the CPU–GPU Divide

> *"The GPU is not a faster CPU—it is a different philosophy of computing."* A CPU makes one runner very fast; a GPU fields a stadium of slower runners and wins by sheer throughput. This chapter explains why that trade exists, how the hardware expresses it, and when each philosophy wins.

From zero: no GPU or parallel background assumed. We start from a single sequential program and ask what happens when we throw thousands of tiny workers at the same problem. Every term—core, SM, warp, SIMT—is defined on first use, picture first, then formal.

Learning Outcomes

After this chapter you will be able to:

(L1) contrast latency-optimised (CPU) and throughput-optimised (GPU) design goals;

(L2) describe the GPU hierarchy (device → SM → warp → thread) and the role of each level;

(L3) define SIMT execution and explain warp-level lockstep, divergence and masking;

(L4) compare SIMD, SIMT and MIMD with examples and state where each excels;

(L5) place a workload on the Roofline and argue CPU versus GPU suitability.

1.1 Why GPUs Exist: Two Philosophies of Speed

Intuition. Imagine delivering 10 000 letters. A CPU is a sports car with a superb navigator: it delivers one letter blazingly fast, with branch prediction guessing turns and a huge cache remembering the map. A GPU is a fleet of 10 000 bicycles that all leave together on the same route: each is slow alone, but the fleet moves the

whole pile in one wave. If your job is one urgent letter, take the car. If it is the whole pile, take the fleet.

A CPU optimises *latency*: minimise time for a single task. It spends transistors on large caches, deep branch predictors, out-of-order scheduling and low-latency control. A GPU optimises *throughput*: maximise tasks per second. It spends transistors on many simple cores, wide memory buses, and hardware that swaps warps instantly to hide stalls. Neither is universally better; they are Pareto-optimal at opposite ends.

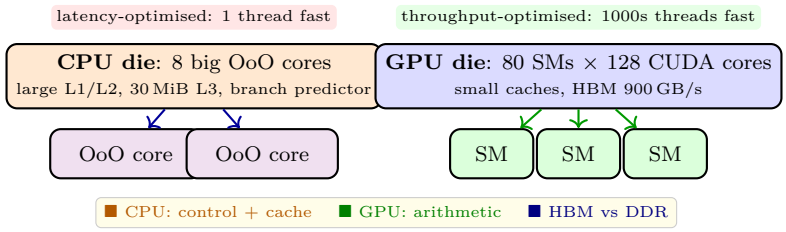


Figure 1.1: CPU versus GPU transistor budget: CPUs buy latency (caches, predictors, OoO), GPUs buy throughput (many simple datapaths and wide HBM).

The same trade appears on the Roofline: a kernel with low arithmetic intensity (bytes per flop) is memory-bound and needs bandwidth (GPU wins); a kernel with branches and irregular dependencies is latency-bound (CPU wins). Knowing the workload's Roofline point is the first step in choosing a device.

1.2 GPU Anatomy: From Device to Thread

A modern NVIDIA GPU (Ampere/Hopper) is a three-level hierarchy.

Device. The whole chip: 60–144 *Streaming Multiprocessors (SMs)*, a shared L2 (40–50 MiB), display/media engines, and high-bandwidth memory (HBM2e/HBM3, 900 GB/s–3 TB/s). The host CPU launches work via PCIe/NVLink.

Streaming Multiprocessor (SM). The workhorse. Each SM holds 64–128 CUDA cores (FP32), 4 warp schedulers, a large register file (64K × 32-bit), L1/shared memory (128–228 KiB configurable), and special units (tensor cores, RT cores, load/store). An SM executes warps, not individual threads, and can keep 32–64 warps resident to hide latency.

Warp and thread. A *warp* is 32 threads that advance in lockstep (SIMT). A *thread* is the smallest unit: one program counter plus private registers. Threads are grouped by the programmer into *blocks* (up to 1024 threads), blocks into a *grid*; the hardware maps blocks to SMs and warps within a block to schedulers.

Definition 1.1 (SM, warp, thread)**.** An *SM* is a self-contained processor that executes warps. A *warp* is 32 threads sharing a program counter and advancing in SIMT lockstep. A *thread* is one SIMT lane with private registers; divergence within a warp is serialised with masking.

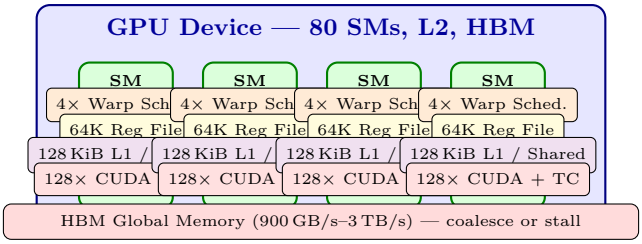


Figure 1.2: GPU SM hierarchy: a device of many SMs, each with warp schedulers, a large register file, configurable L1/shared, and CUDA/tensor cores. Keeping many warps resident hides latency.

1.3 SIMT Execution and Divergence

SIMD (single instruction, multiple data) has one instruction operating on a vector of data in lockstep. *SIMT* looks similar but is subtly different: each thread has its own program counter and can, in principle, diverge. Hardware makes divergence correct but slow: when threads of a warp take different branches, the warp executes each path sequentially with inactive lanes masked off.

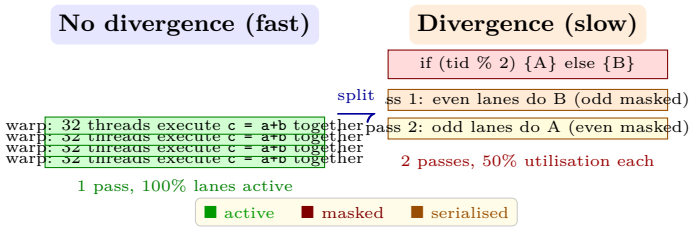


Figure 1.3: SIMT divergence: a warp with no divergence finishes in one pass; a branch that splits the warp executes both paths sequentially with masking, halving throughput.

The rule of thumb is simple: keep warps *converged*. Data-dependent branches where all 32 threads agree are free; branches that split a warp cost proportionally to the number of distinct paths. Sorting or tiling work so that threads in a warp follow the same path is a core GPU optimisation (Ch. 4).

Example 1.1 (Divergence cost)**.** A kernel with `if(data[tid] > 0) { heavyA(); } else { heavyB(); }` and random data splits every warp roughly 50/50, doubling runtime. If data is sorted so warps are homogeneous (all-positive vs. all-negative blocks), each warp takes one path and runtime returns to near-optimal.

Listing 1.1: CPU sequential baseline vs. GPU SIMT kernel: same maths, different execution philosophy.

```
// CPU: one fast thread loops sequentially
void cpuVecAdd(float *a, float *b, float *c, int n){
    for(int i=0;i<n;i++) c[i] = a[i] + b[i]; // latency-optimised core
}
// GPU: thousands of thin threads, one element each, in SIMT lockstep
__global__ void gpuVecAdd(float *a, float *b, float *c, int n){
    int i = blockIdx.x * blockDim.x + threadIdx.x; // global id (Ch.2)
    if(i < n) c[i] = a[i] + b[i]; // warp executes together when converged
}
```

Listing 1.2: Roofline intuition: which device for which kernel?

```
# Roofline: attainable = min(peak_flops, bandwidth * intensity)
peak_cpu, bw_cpu = 1e12, 100e9    # 1 TFLOP, 100 GB/s
peak_gpu, bw_gpu = 15e12, 900e9   # 15 TFLOP, 900 GB/s
```

```
for name, ops, bytes_ in [("saxpy",2,12),("matmul",2,0.25),("branchy",1,8)]:
    intensity = ops/bytes_        # FLOPs per byte
    cpu = min(peak_cpu, bw_cpu*intensity)
    gpu = min(peak_gpu, bw_gpu*intensity)
    print(f"{name:8s} I={intensity:.2f}  CPU {cpu/1e12:.1f} TF  GPU {gpu/1e12:.1f} TF")
# saxpy (low I) is bandwidth-bound -> GPU wins on BW
# matmul (high I) is compute-bound -> GPU wins on peak
# branchy irregular -> CPU wins despite lower peak (divergence)
```

1.4 A Worked Comparison: Latency vs. Throughput in Numbers

Take vector addition of $n = 1\,048\,576$ floats. A 3.5 GHz CPU core with AVX-512 does 16 FLOPs/cycle $\Rightarrow$ 56 GFLOP/s and streams at $\sim$50 GB/s from DDR. A mid-range GPU has 30 SMs $\times$ 128 cores at 1.5 GHz $\approx$ 11 TFLOP/s and 600 GB/s from HBM. The kernel intensity is $I = 2\,\text{FLOPs}/12\,\text{bytes} = 0.167$ FLOP/B (two loads, one store, one add).

Roofline attainable performance is $\min(\text{peak}, \text{BW} \times I)$. On CPU: $\min(56, 50 \times 0.167 = 8.35) = 8.35$ GFLOP/s. On GPU: $\min(11000, 600 \times 0.167 = 100) = 100$ GFLOP/s, a 12$\times$ advantage from bandwidth alone. Yet if the kernel were a binary search with unpredictable branches, SIMT divergence would serialize warps and the CPU's predictor and low latency would dominate despite lower peak.

Remark 1.1 (The golden rule of GPU computing). GPUs reward *uniform, independent, arithmetic-heavy* work. Uniform means warps agree on branches; independent means threads need little cross-talk; arithmetic-heavy (or bandwidth-heavy with coalescing, Ch. 3) keeps the massive datapath busy. Violate any leg and the advantage collapses—that is why CPUs still run operating systems, databases and compilers.

A second lens is *Little's Law* for latency hiding. If global memory latency is $L = 400$ cycles and each warp does $W = 4$ cycles of math before needing data, one warp keeps the SM busy $W/(W + L) \approx 1\%$ of the time. With N warps the SM stays busy $\min(1, N \cdot W/(W + L))$. Solving $N \geq (W + L)/W \approx 101$ warps for 100% utilisation shows why GPUs need *occupancy*: many resident warps cover the gap one warp leaves while waiting.

Definition 1.2 (Arithmetic intensity and Roofline)**.** *Arithmetic intensity* I = FLOPs / bytes moved from DRAM. The *Roofline model* bounds attainable throughput as $\min(\text{peak FLOP/s}, \text{peak BW} \times I)$. The knee where the two terms equal is the *ridge point* $I^* = \text{peak/BW}$; left is memory-bound, right is compute-bound.

What this means for you. Before writing a kernel, ask: is my problem *data-parallel* (same operation on many elements) or *task-parallel* (different operations)? Data-parallel maps to SIMT; task-parallel maps to MIMD (CPU threads or multi-GPU). Within data-parallel, ask: will warps diverge? Profile a histogram of branch outcomes per warp, not per thread. If divergence is high, consider data layout transforms (structure-of-arrays, sorting keys) or algorithmic changes (branchless selects). Finally, estimate intensity early: count FLOPs and bytes in the inner loop. If $I \ll I^*$, optimise memory (coalescing, shared tiling, Ch. 3) before touching arithmetic.

Example 1.2 (Choosing the device)**.** Image blur (3×3 stencil): each output pixel needs 9 multiplies and a handful of loads; warps are fully converged and accesses are regular—ideal GPU work, 8–15$\times$ faster than a multi-core CPU. Binary search in a sorted array: each thread follows a different path length, divergence is

maximal and locality is poor—CPU wins by 5–20×. The deciding factor is not FLOP count but *uniformity of control* and *regularity of memory*.

1.5 Exercises

E1.1 **CPU vs. GPU budgeting.** A chip has 10 B transistors. A CPU core + its cache costs 1 B, an SM costs 0.2 B. How many cores/SMs per design? For a perfectly parallel kernel with 10 000 independent elements, estimate ideal speedup of the GPU design assuming one SM matches one core's throughput on one element. When does the CPU win?

E1.2 **SIMT vs. SIMD vs. MIMD.** Define each and give one real example (e.g., SSE, NVIDIA warp, multicore cluster). Why can SIMT handle `if` divergence (with masking) while pure SIMD cannot handle arbitrary control flow efficiently?

E1.3 **Warp divergence arithmetic.** A kernel branch splits warps 25/75 on average (25% lanes go path A, 75% path B, paths cost equal time T). What is the warp execution time versus a converged warp? If you sort data so warps become 100/0 or 0/100, what is the new time and speedup?

E1.4 **SM occupancy intuition.** An SM supports 2048 threads and 64 warps. Block size is 128 threads (4 warps). How many blocks fit per SM thread-wise? Why might fewer actually fit due to registers or shared memory? No exact numbers needed—explain the limiting idea.

E1.5 **Roofline placement.** A kernel does 5 FLOPs per 4-byte element (intensity 1.25 FLOP/B). CPU ridge point is 10 FLOP/B, GPU ridge is 16.7 FLOP/B (15 TF/900 GB/s). Is it memory or compute bound on each? Which device gives higher attainable performance and by what factor?

Looking ahead. Chapters 2–4 turn this picture into practice. Chapter 2 gives you the programming model that maps threads to SMs; Chapter 3 shows how memory layout makes or breaks the Roofline you just learned to read; Chapter 4 teaches you to measure occupancy and overlap and to know when the GPU is truly busy. Keep the two-philosophies figure in mind—every optimisation is about pushing a workload toward the throughput peak or recognising when latency matters more.

Remark 1.2 (How to read the rest of this book). Each chapter follows the same arc: intuition and picture → formal definition → worked example with numbers → runnable code you can compile with `nvcc -arch=sm_80` → exercises. Run every listing; change a parameter (block size, data layout) and re-measure. The intuition tells you what to expect; the measurement tells you whether you got it. That loop is the core skill of GPU programming.

Example 1.3 (Mental model check)**.** Before moving on, test your model: a kernel launches 1 M threads each doing `c[i]=a[i]+b[i]`. (a) How many warps? (b) If one SM holds 64 warps, how many wavefronts to cover the grid on a 40-SM GPU? (c) Where is the bottleneck—ALUs or memory? Answers: (a) $1\,048\,576/32 = 32\,768$ warps. (b) $32\,768/(40 \times 64) \approx 12.8$ waves. (c) Memory—each thread moves 12 bytes for 1 FLOP, so bandwidth dominates. This back-of-envelope predicts the Roofline result.

Checkpoint. You should now picture a GPU as a grid of SMs each juggling dozens of warps, with warps as the scheduling quantum and threads as lanes. The next chapter turns that picture into code: how you name threads, group them, and launch a kernel that the hardware maps to this hierarchy.

Chapter Notes

Lamport's definition and early GPU history are covered in Kirk & Hwu, *Programming Massively Parallel Processors* (4th ed., 2022, Ch. 1–2) and Hennessy & Patterson, *Computer Architecture: A Quantitative Approach* (6th ed., App. A on GPUs). SIMT terminology originates with NVIDIA's Tesla architecture (Lindholm et al., 2008). Roofline: Williams et al., CACM 2009. CUDA C++ Programming Guide (NVIDIA, v12.x) §1–2 is the authoritative device/SM description; verify SM counts and HBM figures against the specific GPU you target (A100/H100 tables).

Chapter 2

CUDA Threads, Blocks, Grids and Kernels

"A kernel is a single program executed by thousands of threads, each knowing who it is." The CUDA hierarchy—thread, block, grid—is how you name those threads and map them to data. Master the mapping and every kernel becomes bookkeeping; miss it and you read the wrong element or launch the wrong count.

From zero: no CUDA background assumed. We build from a sequential loop to a parallel kernel step by step: what `__global__` means, why we need blocks and grids, how `threadIdx` and `blockIdx` give a global identity, and how warps (Ch. 1) map under the hood. Every index formula is derived, not memorised.

Learning Outcomes

After this chapter you will be able to:

(L1) write and launch a CUDA kernel with correct `___global___` and `«<grid,block»>` syntax;

(L2) compute 1-D and 2-D global thread indices from `threadIdx`, `blockIdx`, `blockDim`, `gridDim`;

(L3) explain the block–SM mapping, why block size matters, and the limits (1024 threads/block, 32 warps/SM);

(L4) use `__syncthreads()` correctly and state what it does and does not synchronise;

(L5) size grids with the ceil division $(n + B - 1)/B$ and guard out-of-bounds accesses.

2.1 From Loop to Kernel: The Launch Hierarchy

Intuition. A sequential loop says "for $i = 0$ to $n - 1$ do $c[i] = a[i] + b[i]$" and one thread walks i step by step. A CUDA kernel flips this: it says "I am thread i; I do $c[i] = a[i] + b[i]$ for my i" and a million threads run that sentence simultaneously, each with a different i. The hierarchy tells each thread what its i is.

CUDA exposes three levels. A *thread* executes one instance of the kernel. Threads are grouped into a *block* (up to 1024 threads, 1-D/2-D/3-D shape) that runs on one SM and can cooperate via shared memory and

`__syncthreads()`. Blocks are grouped into a *grid* (1-D/2-D/3-D) that spans the whole device; blocks are independent and may run in any order on any SM. The programmer chooses block shape; the hardware maps blocks to SMs.

Definition 2.1 (Kernel, block, grid)**.** A *kernel* is a function marked `__global__` executed N times in parallel by N threads. A *block* is a cooperating group of threads (max 1024) on one SM. A *grid* is the collection of blocks launched for one kernel. Built-ins `threadIdx`, `blockIdx`, `blockDim`, `gridDim` give position and size at each level.

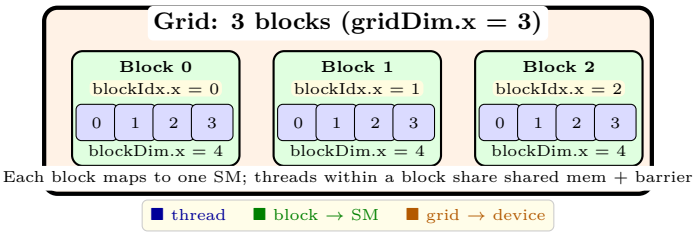


Figure 2.1: Thread hierarchy: 12 threads organised as ⟨grid 3 blocks⟩ × ⟨block 4 threads⟩. Warps cut across threads (here 1 warp per block for illustration; real warps are 32 threads).

Launching uses triple chevrons: `kernel<<<gridDim, blockDim>>>(args)`. Both `gridDim` and `blockDim` may be `int`, `dim3(x)`, or `dim3(x,y,z)`. Inside, `threadIdx.x` ranges 0…`blockDim.x`−1, `blockIdx.x` ranges 0…`gridDim.x`−1. The next section derives the global index.

2.2 Indexing: Who Am I?

Intuition. Mailboxes are numbered globally but you only know your street and house number. Global thread id is the mailbox: `blockIdx`×`blockDim` + `threadIdx`. For 2-D data (images, matrices) the same idea extends to rows and columns.

For 1-D, with `blockDim.x`=256:

$$\texttt{tid} = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x}. \tag{2.1}$$

For 2-D, with blocks of $B_x \times B_y$ and grid $G_x \times G_y$:

$$x = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x}, \tag{2.2}$$

$$y = \texttt{blockIdx.y} \times \texttt{blockDim.y} + \texttt{threadIdx.y}, \tag{2.3}$$

$$\texttt{globalRowMajor} = y \times \texttt{width} + x. \tag{2.4}$$

If total threads exceed n, guard with `if(tid < n) return;`—otherwise out-of-bounds reads corrupt memory silently.

Example 2.1 (Ceiling division)**.** To cover $n = 1000$ elements with $B = 256$ threads/block, need $G = \lceil n/B \rceil = (1000 + 255)/256 = 4$ blocks (1024 threads). Threads 1000–1023 exist but do no work due to the `if(tid < n)` guard. For 2-D image $W = 1920, H = 1080$ with 16×16 blocks, $G_x = \lceil 1920/16 \rceil = 120$, $G_y = \lceil 1080/16 \rceil = 68$.

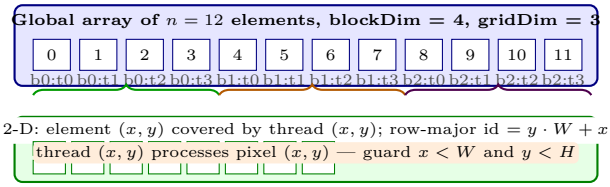


Figure 2.2: Indexing: 1-D global id as block×blockDim + thread; 2-D extends to (x, y) with row-major linearisation. The brace grouping shows how 12 elements map to 3 blocks of 4.

2.3 Warps, Scheduling and Synchronisation

Hardware groups threads into warps of 32 consecutive `threadIdx.x` values (then wraps to next row for 2-D). Warps are the scheduling unit on the SM (Ch. 1). A warp scheduler picks a ready warp each cycle; if one warp stalls on memory, another runs—latency hiding. This is why occupancy (many resident warps, Ch. 4) matters even when indexing looks correct.

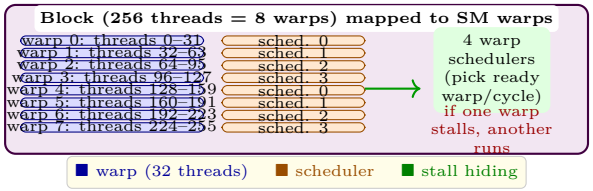


Figure 2.3: Warps within a block: 256 threads become 8 warps; 4 schedulers pick among ready warps each cycle. Thread-to-warp mapping is fixed: consecutive threadIdx group into warps.

Synchronisation. `__syncthreads()` is a block-wide barrier: no thread passes it until all threads in the block have arrived. It is essential when threads share data via shared memory (Ch. 3): write, barrier, then read. It does *not* synchronise across blocks—blocks are independent; for grid-wide sync use `cudaDeviceSynchronize()` on the host or cooperative groups. Placing `__syncthreads()` inside divergent branches causes deadlock (some threads never arrive); keep barriers uniform.

Remark 2.1 (Common indexing bugs). Off-by-one in ceil division (n/B instead of $(n + B - 1)/B$) misses the tail. Forgetting the guard writes past the array. Swapping `blockIdx` and `threadIdx` scrambles order. Mixing `blockDim.x` with `gridDim.x` gives the wrong stride. When in doubt, print `tid` for a tiny n and check the mapping on paper before scaling.

Example 2.2 (Launch configurator pattern)**.** Wrap ceil division in a helper: `dim3 ceilDiv(dim3 n, dim3 B){return {(n.x+B.x-1)/B.x,(n.y+B.y-1)/B.y};}`. Then `grid = ceilDiv({W,H},{16,16})`. This avoids copy-pasted arithmetic and makes 1-D/2-D/3-D consistent. Always assert `grid.x*block.x >= n` in debug builds—a cheap check that catches the most common launch bug.

Listing 2.1: Complete 1-D vector addition: device kernel, host launch with ceil division and guard.

```
__global__ void vecAdd(const float *a, const float *b, float *c, int n){
    int tid = blockIdx.x * blockDim.x + threadIdx.x; // global 1-D id
    if(tid < n) c[tid] = a[tid] + b[tid];             // guard tail
}
int main(){
    int n = 1<<20; // 1M elements
    float *d_a,*d_b,*d_c;
```

```
    cudaMalloc(&d_a, n*sizeof(float));
    cudaMalloc(&d_b, n*sizeof(float));
    cudaMalloc(&d_c, n*sizeof(float));
    int B = 256;                      // threads per block (tunable, Ch.4)
    int G = (n + B - 1)/B;            // ceil(n/B) blocks
    vecAdd<<<G, B>>>(d_a, d_b, d_c, n);
    cudaDeviceSynchronize();          // wait for grid to finish
    // cudaMemcpy back, check, free ...
}
```

Listing 2.2: 2-D kernel: RGB image grayscale. Each thread handles one pixel (x, y).

```
__global__ void rgb2gray(unsigned char *img, unsigned char *gray, int W, int H){
    int x = blockIdx.x * blockDim.x + threadIdx.x; // column
    int y = blockIdx.y * blockDim.y + threadIdx.y; // row
    if(x >= W || y >= H) return;                   // 2-D guard
    int idx = y * W + x;                           // row-major
    unsigned char r = img[3*idx], g = img[3*idx+1], b = img[3*idx+2];
    gray[idx] = (unsigned char)(0.299f*r + 0.587f*g + 0.114f*b);
}
int main2(){
    int W=1920, H=1080;
    dim3 block(16,16); // 256 threads/block
    dim3 grid((W+15)/16, (H+15)/16); // 120 x 68
    // rgb2gray<<<grid, block>>>(d_img, d_gray, W, H);
    return 0;
}
```

2.4 Exercises

E2.1 **Index arithmetic.** With `blockDim.x`=128 and `blockIdx.x`=5, `threadIdx.x`=17, what is `tid`? If $n = 700$ and $B = 128$, compute G and list which `tid` values are guarded out. What happens if you forget the guard?

E2.2 **2-D launch.** An image is 640×480, block is 16×16. Compute G_x, G_y and total threads launched. How many threads are idle? Write the 2-D global (x, y) and linear index formulas.

E2.3 **Block size limits.** Why cannot a block have 2048 threads even if the SM holds 2048? Explain the 1024 limit and give two reasons a smaller block (e.g., 256) is often faster despite launching more blocks.

E2.4 **Barrier correctness.** A kernel does: `shared[tid]=data[tid]; if(tid%2==0) __syncthreads(); y=shared[tid+1];`. Why can this deadlock or give wrong answers? Fix it and explain where `__syncthreads()` must be placed.

E2.5 **Warp mapping.** A block is 32×8 (256 threads, `threadIdx` 2-D). Which threads belong to warp 0, warp 1? Does 2-D thread layout affect warp grouping? Explain the linearisation order assumed by hardware.

Chapter Notes

CUDA C++ Programming Guide (NVIDIA, v12.x) §2–3 defines kernels, hierarchy and built-ins precisely; the `dim3` type and occupancy calculator are documented there. Kirk & Hwu Ch. 3–4 work many indexing examples. For grid-wide synchronisation see Cooperative Groups (`cooperative_groups::grid_group`). Hwu et al., *Heterogeneous System Architecture* (Morgan Kaufmann, 2015) contrasts CUDA with OpenCL/HIP hierarchies (work-group/work-item map to block/thread).

Chapter 3

GPU Memory: Global, Shared, Constant, Coalescing and Bank Conflicts

"Flops are cheap, data movement is expensive." A GPU can do thousands of multiplies per nanosecond but waits hundreds of cycles for one uncoalesced DRAM access. Understanding where data lives and how warps fetch it is the difference between 10% and 90% of peak bandwidth.

From zero: no GPU memory background assumed. We start from the idea that memory is a hierarchy—registers, caches, DRAM—and build the GPU version: global (big, slow), shared (tiny, fast, programmer-managed), constant (broadcast), plus the two performance laws that govern them: coalescing for global and bank conflicts for shared.

Learning Outcomes

After this chapter you will be able to:

(L1) name the GPU memory spaces, their sizes, lifetimes and who manages them;

(L2) explain coalesced versus strided global access and quantify the bandwidth impact;

(L3) describe shared-memory banks, why conflicts serialise, and how to pad or permute to avoid them;

(L4) choose appropriate memory (global, shared, constant, registers) for a given access pattern;

(L5) apply tiling with `__syncthreads()` to exploit shared memory correctly.

3.1 The GPU Memory Hierarchy

Intuition. Think of a building site. Registers are tools in your hands (1 cycle, private to a thread). Shared memory is the workbench on your floor (20–30 cycles, shared by a block, you control). L1/L2 are the site store (80–200 cycles, hardware-managed). Global memory (HBM/DRAM) is the warehouse across town (300–500

cycles, visible to all). Constant memory is a loudspeaker: one value broadcast to every thread efficiently when all read the same address.

Space	Size	Latency	Scope	Managed by
Registers	64K × 32-bit / SM	1	thread-private	compiler
Shared (scratchpad)	48–164 KiB / SM	20–30	block	programmer
L1 / L2 cache	128 KiB / SM, 40–50 MiB dev.	30 / 200	SM / device	hardware
Global (HBM)	16–80 GiB	300–500	device + host	programmer
Constant	64 KiB	5 (hit)	device (broadcast)	programmer

Table 3.1: GPU memory spaces at a glance. Latencies are approximate (Ampere/Hopper). Shared and registers are the only spaces where you control placement explicitly.

Definition 3.1 (Global, shared, constant)**.** *Global* memory is large off-chip DRAM, allocated with `cudaMalloc`, visible to all threads and the host. *Shared* memory (`__shared__`) is on-chip SRAM partitioned per SM, visible only within a block, lifetime = block, explicit barrier needed. *Constant* memory (`__constant__`) is read-only, cached, optimised for uniform reads where all threads in a warp read the same address simultaneously.

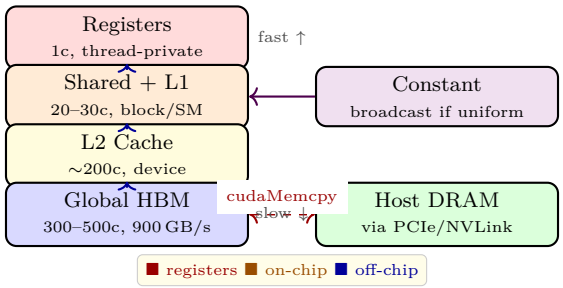


Figure 3.1: GPU memory hierarchy: small fast memories at the top (registers, shared/L1), large slow HBM at the bottom. Constant broadcasts; host link is the narrowest pipe.

3.2 Global Memory and Coalescing

Global accesses move in 32-, 64- or 128-byte *sectors* (cache lines). When 32 threads of a warp access contiguous 4-byte floats at `base + tid*4`, the hardware coalesces them into one or two sectors—full bandwidth. When they access strided (`base + tid*32*4`) or random addresses, the same 32 requests scatter across 32 sectors—you fetch 32× more bytes than needed and waste 97% of bandwidth.

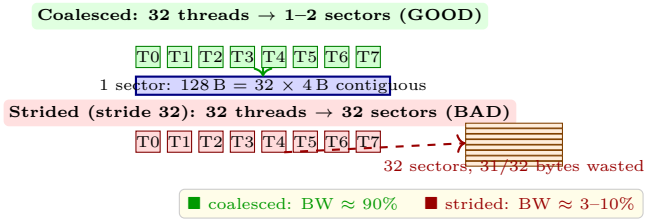


Figure 3.2: Coalescing: contiguous warp accesses collapse to one transaction; strided/random accesses explode to many. Structure-of-arrays and transposed layouts exist to restore coalescing.

The fix is usually a data layout change. Array-of-structures (AoS) `{x,y,z}` per thread is strided when threads read only `x`; structure-of-arrays (SoA) `x[], y[], z[]` is coalesced. Transposing a matrix before a column-wise kernel has the same effect. Unified Memory (`cudaMallocManaged`) hides the copy but not the coalescing cost—the access pattern still governs performance.

Example 3.1 (SoA vs. AoS)**.** 100K particles with `struct {float x,y,z;} p[N];` and kernel `p[tid].x += 1;` is strided (stride 12 B). Rewriting as `float x[N],y[N],z[N];` and `x[tid]+=1;` is contiguous and typically 4–8× faster despite identical maths.

3.3 Shared Memory and Bank Conflicts

Shared memory is split into 32 *banks*, each 4 bytes wide, with successive 4-byte words in successive banks (round-robin). If 32 threads of a warp access 32 different banks, all serve in one cycle. If two or more threads hit the same bank with different addresses (a *bank conflict*), accesses serialise—k-way conflict costs k cycles. Broadcast (all threads read the same address) is free.

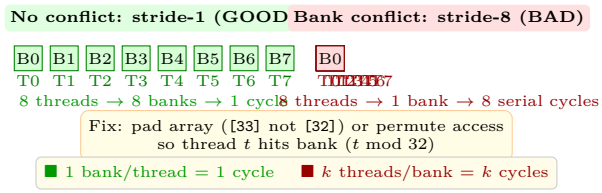


Figure 3.3: Shared-memory banks: contiguous access hits distinct banks (fast); power-of-two strides collapse onto one bank (conflict). Padding breaks the stride.

Classic fix: for a tile `float tile[32][32]`, column access `tile[threadIdx.x][*]` has stride 32 and conflicts; declare `tile[32][33]` to skew banks so each row starts one bank offset. Alternatively, have warps cooperatively load in coalesced/shared-friendly order, `__syncthreads()`, then compute.

Constant and unified memory. Constant memory shines when a warp reads one address together: the single value is fetched once and broadcast, not 32 separate loads. Mark read-only tables bigger than 64 KiB with `__restrict__ const` and let the L1 handle them. Unified Memory (`cudaMallocManaged`) gives a single pointer for host and device and migrates pages on demand—convenient for prototyping and oversubscription, but the same coalescing and bank rules apply after migration. Prefetch with `cudaMemPrefetchAsync` and advise with `cudaMemAdvise` to avoid page-fault storms.

Remark 3.1 (Quick checklist before you optimise). (1) Is global access coalesced? Check with Nsight Compute's "Memory Chart" or count sectors per warp. (2) Is shared access bank-conflict-free? Nsight's "Shared Memory" counter shows conflict rate. (3) Are you using the right space? A common mistake is putting a broadcast table in global instead of constant, or spilling registers to local memory by using too many per thread—both visible in `ptxas` spill reports (`-Xptxas -v`).

Listing 3.1: Coalesced vs. strided global access: same work, very different bandwidth.

```
__global__ void coalesced(float *a, float *b, int n){
    int tid = blockIdx.x*blockDim.x + threadIdx.x;
    if(tid<n) b[tid] = a[tid];                // contiguous: 1 sector/warp
}
__global__ void strided(float *a, float *b, int n, int stride){
    int tid = blockIdx.x*blockDim.x + threadIdx.x;
    if(tid<n) b[tid*stride] = a[tid*stride]; // stride*4 bytes apart: many sectors
}
// Measure: coalesced ~700 GB/s, strided(32) ~20 GB/s on same GPU.
```

Listing 3.2: Shared-memory tiling for 1-D stencil: coalesced load, no bank conflicts, barrier.

```
__global__ void stencil1D(float *in, float *out, int n){
    __shared__ float tile[256+2]; // block + halo (padding avoids conflict when needed)
    int tid = blockIdx.x*blockDim.x + threadIdx.x;
    int lid = threadIdx.x;
    // coalesced global -> shared
    tile[lid+1] = (tid<n) ? in[tid] : 0;
    if(lid==0) tile[0] = (tid>0) ? in[tid-1] : 0;
    if(lid==blockDim.x-1) tile[lid+2] = (tid+1<n) ? in[tid+1] : 0;
    __syncthreads(); // all loads done before compute
    if(tid<n) out[tid] = 0.25f*tile[lid] + 0.5f*tile[lid+1] + 0.25f*tile[lid+2];
    // no __syncthreads needed after: no cross-thread reuse
}
```

Example 3.2 (End-to-end: matrix transpose with and without shared tiling)**.** Naive transpose `out[j*W+i]=in[i*H+j]` is coalesced on read but strided on write (or vice versa). A tiled version loads a 32×32 tile into shared with coalesced reads, `__syncthreads()`, then writes transposed from shared to global—both sides coalesced and bank-conflict-free with padding [32][33]. The tiled kernel is typically 3–7× faster on large matrices, a textbook win from two ideas in this chapter composed together.

Remark 3.2 (Nsight check). Profile the transpose pair with `ncu -metrics l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum,l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum`. The naive shows ∼32× sectors per warp on one side; the tiled shows ∼1–2 on both. The ratio matches your wall-clock speedup—when counters and clocks agree, you understand the bottleneck.

Takeaway. Global optimisation is about addresses: make warps contiguous. Shared optimisation is about banks: make threads hit distinct banks. Both are layout problems, not maths problems—you often fix them by transposing data or padding an array, not by changing FLOPs.

3.4 Exercises

E3.1 **Memory choice.** For each pattern, pick the best space (registers, shared, constant, global) and justify: (a) per-thread private accumulator, (b) 64 floats reused by all threads in a block 10 times, (c) a 4 KiB lookup table where every thread in a warp reads the same entry, (d) a 100 MiB input streamed once.

E3.2 **Coalescing arithmetic.** A warp reads 4-byte floats. Contiguous needs 1 × 128 B sector. Stride-8 needs how many sectors? If peak is 900 GB/s, estimate attained BW for each. How does SoA rescue AoS here?

E3.3 **Bank conflict.** Shared memory has 32 banks. A kernel has 32 threads each read `s[tid*4]` (stride 4). How many distinct banks are touched? How many-way conflict? Propose a padding or indexing fix and compute new bank mapping.

E3.4 **Tiling correctness.** In the stencil listing, why are there two `if(lid==0)` halo loads? What breaks if `__syncthreads()` is removed? What breaks if it is placed inside `if(tid<n)`?

E3.5 **Constant vs. global.** Kernel reads `params[k]` where `k` is uniform across a warp 90% of the time but

divergent 10%. When does constant memory win? When does it lose to global with L1? Quantify broadcast benefit versus cache-miss cost.

Chapter Notes

Memory spaces and qualifiers are specified in CUDA C++ Programming Guide §5–7; bank width and count (32×4 B) confirmed for Ampere/Hopper—verify for older compute capabilities. Coalescing rules (sector granularity) are in the Guide's Global Memory section and Volkov, *Better performance at lower occupancy* (GTC 2010). Shared-memory tiling is covered in Kirk & Hwu Ch. 5 and Ryoo et al., "Optimization principles and application performance evaluation of a multithreaded GPU" (PPoPP 2008). Constant-cache broadcast is in Guide §5.3.3.

Chapter 4

Optimization: Occupancy, Latency Hiding and CUDA Streams

> *"You cannot make memory faster, but you can make sure the GPU always has something else to do while waiting."* Occupancy tells you how many warps are available to do that something; streams let you keep the whole system—copy engines, compute, and CPU—busy at once.

From zero: assumes Ch. 1–3. We know warps, blocks and memory. This chapter asks: how many warps can an SM hold? How does that hide latency? And how do we overlap work that would otherwise serialize? No queuing theory assumed; we derive everything from Little's Law and a picture.

Learning Outcomes

After this chapter you will be able to:

(L1) define occupancy and compute it from registers, shared memory and thread limits;

(L2) explain latency hiding via ready warps and why higher occupancy is not always faster;

(L3) use Nsight Compute / Occupancy Calculator to spot the limiting resource;

(L4) create CUDA streams and overlap host-to-device copy, kernel, and device-to-host copy;

(L5) choose block size and launch configuration with a quantitative justification.

4.1 Occupancy: How Many Warps Fit?

Intuition. An SM is a hotel: rooms are warps, guests are threads, and amenities are registers and shared memory. Occupancy is the fraction of rooms occupied. A full hotel can always send another guest to the restaurant while one waits for laundry; an empty hotel cannot—the single guest stalls everything.

Formally, for one SM:

$$\text{occupancy} = \frac{\text{active warps}}{\text{max warps per SM}}. \tag{4.1}$$

Max warps is hardware (e.g., 64 on Ampere, i.e., 2048 threads). Active warps is the minimum of four caps: (1) thread cap: $\lfloor 2048/B \rfloor \times (B/32)$ warps from B threads/block; (2) block cap: max 32 blocks/SM; (3) register cap: $\lfloor$regs per SM/regs per warp$\rfloor$; (4) shared-memory cap: $\lfloor$shared per SM/shared per block$\rfloor \times (B/32)$. The smallest wins. Increasing B or reducing per-thread registers/shared often raises occupancy—until another cap binds.

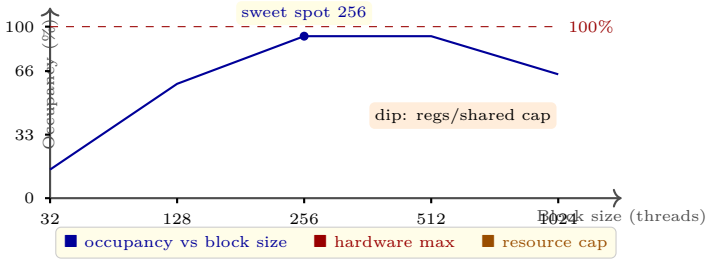


Figure 4.1: Occupancy versus block size for a typical kernel: rises as more warps fill the SM, plateaus at the hardware max, then dips when registers or shared memory per block limit resident blocks.

Definition 4.1 (Occupancy limiters)**.** *Register pressure* is registers per thread × threads per block exceeding the SM file. *Shared-memory pressure* is shared bytes per block × blocks per SM exceeding the SM budget. Either forces fewer resident blocks and lower occupancy even if threads alone would allow more.

Example 4.1 (Occupancy calculation)**.** SM: 2048 threads, 64K registers, 100 KiB shared. Kernel: $B = 256$ threads (8 warps), 32 regs/thread, 4 KiB shared/block. Thread cap: $2048/256 = 8$ blocks $\to$ 64 warps. Reg cap: $65536/(32 \times 256) = 8$ blocks $\to$ 64 warps. Shared cap: $100/4 = 25$ blocks but thread cap already 8, so 8 blocks $\to$ 64 warps $\to$ 100%. If regs rise to 64/thread, reg cap drops to 4 blocks $\to$ 32 warps $\to$ 50% occupancy.

4.2 Latency Hiding: Why Occupancy Matters (and When It Does Not)

Memory latency L is 300–500 cycles; a warp that issues a global load is not ready for L cycles. If the SM has W warps and each does C cycles of math per load, Little's Law says the SM stays busy when $W \geq L/C + 1$. Doubling W (higher occupancy) doubles the chance a ready warp exists when one stalls.

But occupancy is not performance. A kernel with high instruction-level parallelism (ILP) or many independent instructions per thread can hide latency with few warps. Conversely, pushing occupancy by spilling registers to "local" memory (which is DRAM) hurts more than the extra warps help. The correct metric is wall time, not occupancy; use occupancy as a diagnostic, not a goal. Nsight Compute reports "SM Throughput" and "Memory Throughput" alongside theoretical and achieved occupancy—trust those.

Practical tuning. Start with $B = 128$ or 256 (multiple of 32, enough warps, not too much shared). Check `nvcc -Xptxas -v` for regs/thread and shared bytes. Plug into the occupancy calculator (spreadsheet or `https://xmartlabs.github.io/cuda-occupancy-calculator`) to see the limiter. If shared or regs cap, try `__launch_bounds__` or split the kernel. Measure before and after—10% higher occupancy that costs a spill is a loss.

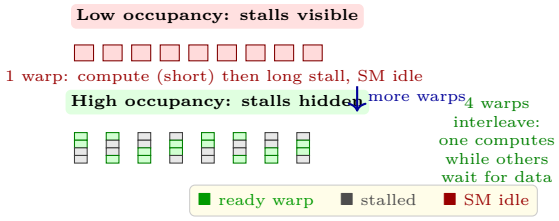


Figure 4.2: Latency hiding: with one warp the SM idles during loads; with four warps another ready warp fills each gap. More ready warps ⇒ higher utilisation, until math or bandwidth saturates.

4.3 CUDA Streams: Overlapping Copy and Compute

So far kernels and copies were serial: host-to-device (H2D) → kernel → device-to-host (D2H), with the copy engine and compute engine idle in turn. *Streams* are ordered queues; operations in one stream are sequential, but streams run concurrently when hardware allows. With two copy engines (H2D and D2H) plus compute, a three-stage pipeline can overlap.

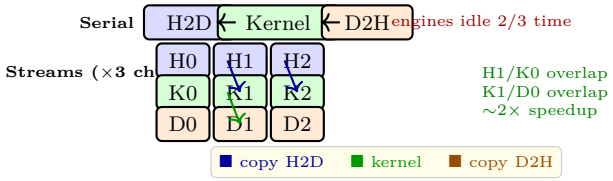


Figure 4.3: Streams overlap: serial execution leaves engines idle; chunking data into N pieces and pipelining H2D/Kernel/D2H across streams keeps copy and compute busy together.

Requirements for overlap: pinned (page-locked) host memory (`cudaMallocHost` or `cudaHostAlloc`) for async copies, and async APIs (`cudaMemcpyAsync`, kernel launch). Default stream (0) synchronises with others unless created with `cudaStreamNonBlocking`. Events (`cudaEventRecord` / `cudaStreamWaitEvent`) express dependencies without host sync.

Listing 4.1: Occupancy-aware launch helper and register report.

```
#include <cuda_runtime.h>
// Ask compiler to limit registers to raise occupancy (may spill if too low)
__global__ void __launch_bounds__(256, 4) myKernel(float *a, int n){
    int tid = blockIdx.x*blockDim.x + threadIdx.x;
    if(tid<n) a[tid] = a[tid]*2.0f + 1.0f;
}
int main(){
    int B=256; int n=1<<20;
    // compile with: nvcc -Xptxas -v -> reports regs/thread, shared, occupancy
    int minGrid, blockSize;
    cudaOccupancyMaxPotentialBlockSize(&minGrid, &blockSize, myKernel, 0, 0);
    // blockSize is heuristic optimum; measure around it (128,256,512)
    return 0;
}
```

Listing 4.2: Overlapping copy and compute with two streams and pinned memory.

```
int N=1<<24; size_t bytes=N*sizeof(float);
float *h_a, *d_a;
cudaMallocHost(&h_a, bytes);      // pinned: async copies can overlap
cudaMalloc(&d_a, bytes);
```

```
cudaStream_t s0,s1;
cudaStreamCreate(&s0); cudaStreamCreate(&s1);
int chunk = N/2;
for(int i=0;i<2;i++){
    float *h_chunk = h_a + i*chunk;
    float *d_chunk = d_a + i*chunk;
    cudaMemcpyAsync(d_chunk, h_chunk, chunk*sizeof(float),
                    cudaMemcpyHostToDevice, i==0?s0:s1);
    myKernel<<< (chunk+255)/256, 256, 0, i==0?s0:s1 >>>(d_chunk, chunk);
    // D2H similarly; use cudaEventRecord / cudaStreamWaitEvent for deps
}
cudaDeviceSynchronize(); // wait for both streams
cudaFreeHost(h_a); cudaFree(d_a);
```

When streams help. Overlap hides H2D/D2H time only if kernel time $\approx$ copy time; if the kernel dominates, overlap is minor. Chunking also improves load balance across SMs. For multi-GPU, streams plus `cudaMemcpyPeerAsync` overlap inter-GPU transfers similarly (preview of Ch. 7).

4.4 Exercises

E4.1 **Occupancy math.** SM: 2048 threads, 64K regs, 96 KiB shared, max 32 blocks. Kernel A: $B = 128$, 32 regs/thread, 2 KiB shared/block. Kernel B: $B = 256$, 48 regs/thread, 8 KiB shared/block. Compute occupancy for each and name the limiter. Which would you tune first?

E4.2 **Latency hiding estimate.** Global latency $L = 400$ cycles, each thread does $C = 8$ math cycles per load, 4 warps resident. Is the SM fully utilised? How many warps needed for 100%? If you double C via ILP (unrolling), how does the requirement change?

E4.3 **Block-size sweep.** You measure kernel time for $B = 32, 64, 128, 256, 512, 1024$ and get 12, 8, 6, 5.5, 6.2, 7.1 ms. Explain the U-shape: why does tiny B hurt, why does huge B hurt despite higher occupancy? What would you pick and why?

E4.4 **Streams correctness.** Two streams copy chunks 0 and 1 and launch kernels K0, K1 that read their chunk. Without events, can K1 start before H2D of chunk 1 finishes? Can K0 read chunk 1? Draw the dependency graph and add one event to enforce "H1 before K1" without serialising streams.

E4.5 **Pinned memory.** Why does `cudaMemcpyAsync` require pinned host memory to overlap? What happens if you use pageable memory or forget `cudaStreamNonBlocking`? Quantify: pageable copy pins on the fly at $\sim$6 GB/s vs. pinned at $\sim$25 GB/s PCIe4—how does this affect overlap benefit?

Putting it together. A good optimisation workflow is: (1) profile baseline with Nsight Systems (see batch vs. stream timeline, copy/compute overlap). (2) Check Nsight Compute occupancy and limiter. (3) Fix the limiter (adjust B, reduce regs/shared, or accept lower occupancy if ILP is high). (4) Chunk work and add streams where copy time matters. (5) Re-measure; if occupancy rose but time did not, you chased the wrong metric—go back to the Roofline.

Remark 4.1 (Rules of thumb that survive). Small blocks (64–128) rarely saturate; huge blocks (1024) often exhaust regs/shared and hurt flexibility. 256 is a reliable default—measure one step either side. Pinned memory + two streams + double-buffering is the simplest overlap that wins on most PCIe-bound workloads. For compute-bound kernels, overlap barely helps; spend time on coalescing and bank conflicts (Ch. 3) instead.

Chapter Notes

Occupancy, `__launch_bounds__` and the calculator are documented in CUDA C++ Programming Guide §7 and CUDA Toolkit "Occupancy Calculator" (spreadsheet and runtime API `cudaOccupancy*`). Little's Law treatment follows Volkov, *Better performance at lower occupancy* (GTC 2010) and the "Latency hiding" note in Guide §7. Streams, events and async copies: Guide §3.2.5–3.2.6 and Wilt, *The CUDA Handbook* (2nd ed., 2013, Ch. 7). Profiling: Nsight Compute ("Occupancy", "SM Throughput") and Nsight Systems (stream timeline). For deeper overlap see Ch. 7 on multi-GPU and cooperative groups.

Chapter 5

GPU Libraries: cuBLAS, cuDNN, Thrust, and CUTLASS

Do not rewrite what NVIDIA perfected — learn to stand on 10 000 engineer-hours of tuning.

From zero: no library expertise assumed. We start from a familiar idea: a well-tuned library saves you from hand-writing every loop. On the CPU you call `BLAS` for matrices; on the GPU the same insight holds, but the library must also hide warps, tiling, Tensor Cores, and memory coalescing. First intuition, then APIs.

Learning Outcomes

After this chapter you will be able to:

(L1) explain why GPU libraries outperform naive kernels and when to use them;

(L2) call cuBLAS for `gemm` with correct handle, leading dimension, and transpose flags;

(L3) describe cuDNN tensor descriptors and why convolution needs workspace;

(L4) write Thrust code using device vectors and fused transforms;

(L5) outline CUTLASS hierarchical tiling and how it exposes Tensor Core control.

5.1 Why Libraries Beat Hand-Written Kernels

A naive $N \times N$ matrix multiply does $O(N^3)$ work and $O(N^2)$ data movement. The performance gap between a triple loop and a tuned library is 20–100× because the library does: (i) tiling to fit shared memory, (ii) double-buffered async copies, (iii) warp-level matrix instructions, and (iv) auto-tuned tile sizes per architecture. Reimplementing all four is months of work; calling a library is one function.

Think of it as the GPU analogue of `sort()` vs. bubble sort: correctness is easy, near-peak efficiency is not. Libraries also track new hardware — Hopper's `wgmma` appears first in CUTLASS, months before textbook kernels

catch up.

Definition 5.1 (GPU compute library)**.** A pre-compiled, auto-tuned collection of kernels plus a host API that abstracts tiling, data layout, and hardware intrinsics, exposing a stable interface (e.g., `cublasSgemm`) while internally selecting the fastest algorithm for the current GPU, data type, and shape.

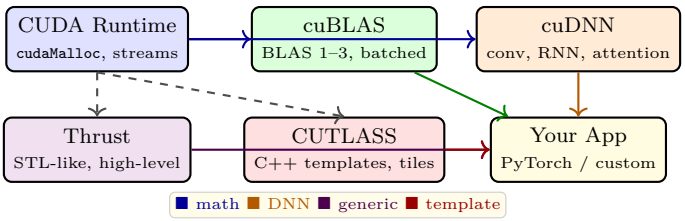
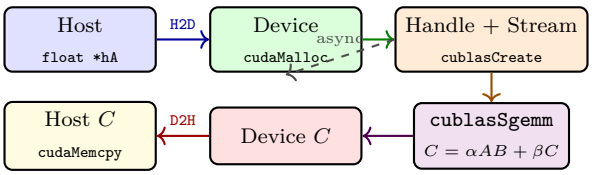


Figure 5.1: GPU library stack. All build on the CUDA runtime; domain libraries (cuBLAS/cuDNN) sit beside generic tools (Thrust/CUTLASS) and compose into applications.

5.2 cuBLAS: The Workhorse for Linear Algebra

BLAS levels: L1 vector ops ($O(n)$), L2 matrix-vector ($O(n^2)$), L3 matrix-matrix ($O(n^3)$). GPU speedups grow with arithmetic intensity, so L3 `gemm` is where cuBLAS shines — often $> 90\%$ of peak.

Key API pattern: create a `cublasHandle_t`, set stream with `cublasSetStream`, then call `cublasSgemm` / `cublasGemmEx`. Column-major is default (Fortran legacy); C row-major users transpose logically via `CUBLAS_OP_T` or set `CUBLAS_LT` order. `lda, ldb, ldc` are leading dimensions, not N — stride between columns.



Batched `gemm` matters for small matrices (e.g., attention heads): one call launches many tiny multiplies with one overhead. `cublasGemmStridedBatchedEx` adds mixed precision (`fp16` in, `fp32` accumulate) to use Tensor Cores automatically.

Listing 5.1: cuBLAS SGEMM with handle, stream, and error check. Column-major $\alpha = 1, \beta = 0$.

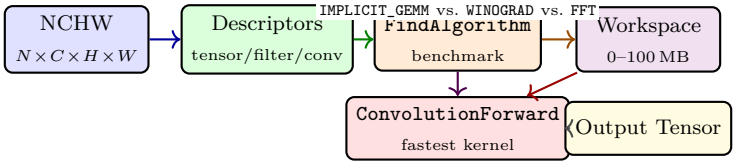
```
#include <cublas_v2.h>
cublasHandle_t h; cublasCreate(&h);
cudaStream_t s; cudaStreamCreate(&s);
cublasSetStream(h, s);
int N=2048; float alpha=1,beta=0;
float *dA,*dB,*dC;
cudaMalloc(&dA,N*N*sizeof(float));
cudaMalloc(&dB,N*N*sizeof(float));
cudaMalloc(&dC,N*N*sizeof(float));
// ... fill dA,dB via cudaMemcpyAsync(..., s)
cublasSgemm(h, CUBLAS_OP_N, CUBLAS_OP_N, N,N,N,
            &alpha, dA,N, dB,N, &beta, dC,N);
cudaStreamSynchronize(s);
cublasDestroy(h);
```

Common pitfall: forgetting `cublasSetMathMode(h, CUBLAS_TENSOR_OP_MATH)` — without it, large gemms may not use Tensor Cores and run 4–8× slower on Ampere+.

5.3 cuDNN: Deep Learning Primitives

cuDNN accelerates convolutions, RNNs, and fused attention. Unlike cuBLAS's flat arrays, cuDNN uses *descriptors*: `cudnnTensorDescriptor_t`, `cudnnFilterDescriptor_t`, `cudnnConvolutionDescriptor_t`. You describe shapes, strides, data type, and layout (`NCHW` vs. `NHWC`); cuDNN picks the fastest algorithm via `cudnnFindConvolutionForwardAlgorithm`.

Why the indirection? Convolution can lower to GEMM (`im2col` + gemm), use FFT, or use Winograd — each wins on different shapes. cuDNN benchmarks candidates autotuning-style during `Find` and caches the choice. Workspace bytes trade memory for speed; `cudnnGetConvolutionForwardWorkspaceSize` tells you how much to allocate.



Fused ops (`cudnnConvolutionBiasActivationForward`) keep intermediates in registers/shared memory, cutting DRAM traffic — the same fusion that drives Transformer speedups.

5.4 Thrust and CUTLASS: Two Ends of Abstraction

Thrust is STL for GPUs: `thrust::device_vector`, `transform`, `reduce`, `sort`. It picks backend (CUDA, TBB, OpenMP) at compile time. Productivity is high; control over tiling is low — fine for preprocessing, not for squeezing the last 30% of matmul.

CUTLASS is the opposite: C++ templates where you compose `ThreadblockTile` → `WarpTile` → `InstructionTile`. You pick tile sizes, pipeline stages, epilogues, and the template elaborates to a kernel using `mma.sync` or Hopper `wgmma`. NVIDIA ships CUTLASS as the reference for new hardware features; PyTorch's fast kernels often are CUTLASS instantiations.

Listing 5.2: Thrust: device vectors, transform, and reduce on the GPU.

```
#include <thrust/device_vector.h>
#include <thrust/transform.h>
#include <thrust/reduce.h>
thrust::device_vector<float> a(1<<20, 1.0f), b(1<<20, 2.0f), c(1<<20);
// c = a + b using a functor (lambda needs __host__ __device__)
thrust::transform(a.begin(), a.end(), b.begin(), c.begin(),
                  [] __host__ __device__ (float x, float y){ return x+y; });
float sum = thrust::reduce(c.begin(), c.end(), 0.0f, thrust::plus<float>());
// sum == 3 * 2^20 ; no explicit kernel, no cudaMalloc
```

Choosing: prototype with Thrust/cuBLAS, fuse with cuDNN, and when roofline says you are still compute-bound and need custom layouts, drop to CUTLASS. The three interoperate — CUTLASS can consume cuBLASLt layouts, and Thrust can wrap raw device pointers via `thrust::device_ptr`.

Remark 5.1. Library version lock is real: cuBLAS 12.x requires CUDA 12.x and driver $\geq$525. Pin versions in Docker, and prefer `cublasLt` / `cudnn` 8+ which decouple heuristics from the binary via JSON plan caches.

5.5 cuBLASLt, Mixed Precision, and Common Pitfalls

`cublasLt` is the lightweight, more explicit cousin of cuBLAS. It exposes *matmul descriptors*, *algorithm heuristics*, and *epilogues* (bias + ReLU) that fuse without extra passes. Instead of one `Sgemm` call, you build a `cublasLtMatmulDesc`, set compute type (`CUBLAS_COMPUTE_32F`), scale type, and let `cublasLtMatmulAlgoGetHeuristic` rank candidates for your exact M, N, K.

Mixed precision is where the win lies. Store in `fp16` or `bf16`, compute in `fp32`, accumulate in `fp32`: `cublasGemmEx` with `CUDA_R_16F` inputs, `CUDA_R_32F` compute. On Ampere+, this hits Tensor Cores automatically if `cublasSetMathMode` or Lt math mode allows it. Hopper `fp8` (E4M3/E5M2) doubles throughput again, but needs per-tensor scaling to avoid overflow — CUTLASS 3.x shows how to set `scaleA, scaleB`.

Example 5.1 (When libraries lose)**.** Libraries assume large, regular shapes. For tiny 16×16 batch, launch overhead dominates and a fused custom kernel that does matmul + activation in registers beats cuBLAS by $2\times$. For sparse matrices ($< 1\%$ nonzero), `cuSPARSE` or block-sparse CUTLASS wins over dense cuBLAS. The roofline tells you: if your AI is low, no GEMM library saves you — restructure the algorithm.

Pitfalls checklist: (1) column-major vs. row-major transposes — profile, not guess; (2) not setting stream — default stream serialises; (3) allocating workspace too small — cuDNN returns `CUDNN_STATUS_NOT_SUPPORTED`; (4) ignoring `cudaDataType` vs. `cublasComputeType` mismatch — silent wrong answers in mixed precision; (5) benchmarking with cold cache — warm up 5 iterations before timing.

Definition 5.2 (Epilogue fusion)**.** Applying element-wise ops (bias, activation, scaling) *inside* the GEMM kernel before writing to DRAM, so data stays in registers/shared memory. CUTLASS expresses this as an `EpilogueOp`; cuBLASLt as `CUBLASLT_EPILOGUE_RELU_BIAS`. Fusion raises AI by removing a memory pass.

Choosing flow: start with cuBLAS/cuDNN for correctness, then try cuBLASLt heuristics, then if profiler still shows Tensor pipe idle or extra passes, prototype a CUTLASS tile. Most teams stop at step two — CUTLASS is for the last 10% and for new hardware day-zero.

5.6 Choosing the Right Level and Versioning

A practical decision tree: (1) Is your op in cuBLAS/cuDNN? Use it. (2) Do you need fusion or custom layout? Try cuBLASLt/cuDNN v8 plan API. (3) Do you need a novel tile or new dtype (fp8, fp4)? Use CUTLASS. (4) Do you need absolute control or a non-GEMM pattern? Write a custom kernel (Ch. 4).

Versioning matters: CUDA 11 vs. 12 changes `cublasComputeType` enums; cuDNN 8 introduced the `cudnnBackend` plan API that deprecates `cudnnFind` for graphs. Pin `cuda-toolkit`, `cublas`, and `cudnn` together in your `Dockerfile` and CI. NVIDIA's NGC containers already do this.

Performance tip: warm up. First call triggers JIT and autotuning (cuDNN `Find` can take seconds). Benchmark the 10th iteration, not the first, and call `cudaDeviceSynchronize` before timing.

5.7 Exercises

Exercise 5.1. Explain why a naive $O(N^3)$ matmul is 20× slower than cuBLAS even though both do $2N^3$ FLOPs. Name three optimisations cuBLAS applies and which roofline effect each has (move up vs. right).

Exercise 5.2. Write cuBLAS code for $C = 1.5\,A^T B + 0.5\,C$ where A, B, C are 1024×1024 row-major. Which `CUBLAS_OP_*` flags and leading dimensions do you need? How do you enable Tensor Core math?

Exercise 5.3. A convolution has N=32, C=64, H=W=56, K=128, R=S=3. Estimate FLOPs and bytes for `im2col`+GEMM lowering. Why does `cudnnFindConvolutionForwardAlgorithm` sometimes pick Winograd instead? When would you prefer the `IMPLICIT_GEMM` algorithm?

Exercise 5.4. Rewrite a Thrust `transform` + `reduce` that computes $\sum_i (a_i + b_i)^2$ as a single `transform_reduce` with a fused functor. Why does fusion improve arithmetic intensity and reduce DRAM passes from two to one?

Exercise 5.5. CUTLASS lets you set `ThreadblockTile` 128×128×32 and `WarpTile` 64×64×32. How many warps per threadblock? How many `mma.sync` instructions per warp per k-iteration for `fp16` 16×8×16 tiles? What breaks if the tile does not divide the problem size?

5.8 Chapter Notes

cuBLAS docs: `docs.nvidia.com/cuda/cublas`; cuDNN 8+ API with `cudnnFind` vs. `cudnnGetPlan`; Thrust GitHub `NVIDIA/thrust` and Bell & Hoberock “Thrust: A Productivity-Oriented Library”; CUTLASS `NVIDIA/cutlass` docs and the GTC “CUTLASS: Fast Linear Algebra” talk; Thall “CUTLASS 3.x and Hopper wgmma” blog. For heuristics, read Volkov & Demmel SC08 GEMM and Jia et al. “Dissecting cuDNN”.

Chapter 6

Multi-GPU: NCCL, NVLink, and Scaling

One GPU is a sports car; eight is a freight train — powerful only if the couplings hold.

From zero: single-GPU thinking first. We scale from one to many GPUs by asking what changes: memory capacity, bandwidth between devices, and how to keep all GPUs busy without drowning in communication. Picture first (topology and collectives), then bandwidth math and NCCL code.

Learning Outcomes

After this chapter you will be able to:

(L1) compare PCIe, NVLink, and NVSwitch bandwidth/latency and choose topology for a workload;

(L2) explain NCCL collectives (all-reduce, broadcast, all-gather) and the ring algorithm;

(L3) distinguish data-parallel, model-parallel, and pipeline-parallel training;

(L4) launch a multi-GPU job with NCCL and overlap comm with compute;

(L5) estimate scaling efficiency via Amdahl plus communication overhead.

6.1 Why One GPU Is Not Enough

Three walls push to multi-GPU: *capacity* (80 GB HBM fits ∼40B params in fp16; GPT-3 needs 350 GB), *throughput* (training ImageNet in hours, not weeks), and *latency* (inference batch size). The fix is P GPUs, but speedup is bounded by communication: if compute per GPU shrinks as $1/P$ but communication stays O(model size), efficiency drops.

The hardware answer is faster links; the software answer is smarter collectives. Both matter: a 900 GB/s NVLink is wasted if your collective sends $P\times$ too much data.

Definition 6.1 (Scaling efficiency)**.** $E(P) = T_1/(P \cdot T_P)$, where T_P is time on P GPUs. $E = 1$ is perfect linear;

$E = 0.7$ means 30% lost to communication, load imbalance, or serial work. Strong scaling fixes problem size; weak scaling grows it with P.

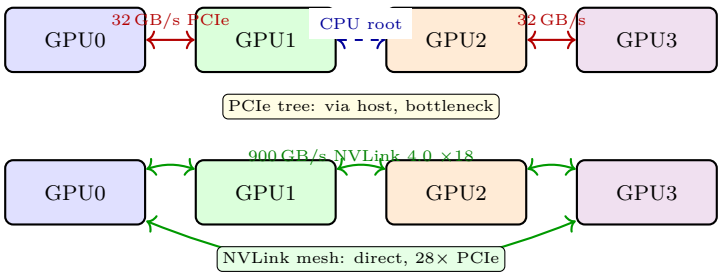


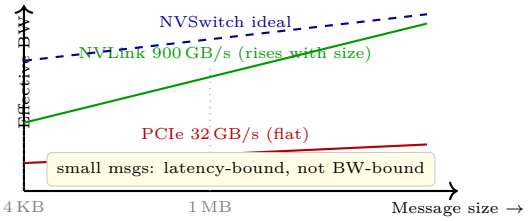
Figure 6.1: Interconnect matters more than FLOPs at scale. PCIe forms a tree through the CPU; NVLink forms a direct mesh. Numbers are bidirectional per GPU for Hopper (NVLink 4.0).

6.2 NVLink, NVSwitch, and Topology

PCIe 5.0 ×16 gives ∼64 GB/s bidirectional per GPU via the host root complex — fine for one GPU, a bottleneck for eight all-reducing 10 GB gradients.

NVLink 4.0 (Hopper) gives 900 GB/s per GPU over 18 links (50 GB/s each, 2× NVLink 3.0). Links connect GPUs directly; a fully connected 8-GPU DGX H100 uses *NVSwitch* — a crossbar that routes any GPU to any GPU at full speed, appearing as a single-hop all-to-all. Without NVSwitch, 8 GPUs form a hybrid cube-mesh where some pairs need two hops.

Topology determines collective bandwidth. A ring needs only neighbour links; a tree needs bisection bandwidth. NVSwitch removes topology sensitivity by giving full bisection.



6.3 NCCL and Collectives: The Ring Insight

NCCL (NVIDIA Collective Communications Library) implements `allReduce`, `broadcast`, `reduce`, `allGather`, `reduceScatter`. It is topology-aware: it probes NVLink/PCIe/InfiniBand and picks ring, tree, or collnet algorithm.

The classic is *ring all-reduce*: P GPUs in a ring, data chunked into P pieces. Phase 1 (scatter-reduce): each GPU sends one chunk clockwise, accumulating for $P-1$ steps. Phase 2 (all-gather): circulate the reduced chunks for $P-1$ steps. Total data per GPU $= 2(P-1)/P \cdot N \approx 2N$, bandwidth-optimal and independent of P for large N.

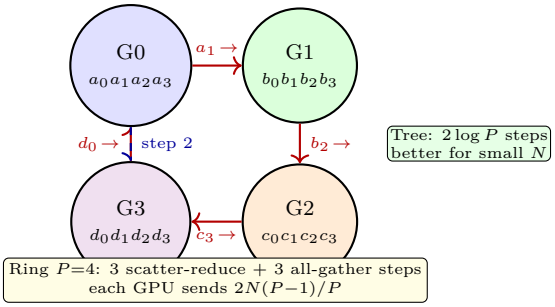


Figure 6.2: Ring all-reduce. Chunks rotate and accumulate; after $2(P-1)$ steps every GPU holds $a+b+c+d$ fully reduced. NCCL picks ring for large N, tree for small.

Latency matters for small messages: ring's $2(P-1)$ hops hurt; tree's $\log P$ wins. NCCL's autotuner benchmarks both.

Overlap is critical: do `ncclAllReduce` on stream S_{comm} while next layer computes on S_{comp}, then `cudaEventSynchronize`. Without overlap, 8 GPUs spend 30–50% of step in communication.

Listing 6.1: NCCL all-reduce across 4 GPUs. One process per GPU, shared communicator.

```
#include <nccl.h>
int rank, world=4; // launched via mpirun -np 4
ncclComm_t comm; ncclCommInitRank(&comm, world, id, rank);
cudaSetDevice(rank);
float *dBuf; cudaMalloc(&dBuf, N*sizeof(float));
// ... compute local gradients into dBuf
cudaStream_t s; cudaStreamCreate(&s);
ncclAllReduce(dBuf, dBuf, N, ncccFloat, ncclSum, comm, s);
cudaStreamSynchronize(s); // now dBuf holds global sum
// divide by world for mean, then optimizer step
```

Listing 6.2: Overlap: compute next layer while reducing current grad. Two streams + event.

```
cudaStream_t compS, commS; cudaStreamCreate(&compS); cudaStreamCreate(&commS);
cudaEvent_t e; cudaEventCreate(&e);
for(int layer=L-1; layer>=0; --layer){
  launchBackward(layer, compS);          // compute dW[layer]
  cudaEventRecord(e, compS);
  cudaStreamWaitEvent(commS, e, 0);      // comm waits for grad ready
  ncclAllReduce(dW[layer], dW[layer], Sz[layer], ncclFloat, ncclSum, comm, commS);
}
cudaStreamSynchronize(commS); cudaStreamSynchronize(compS);
```

6.4 Data, Model, and Pipeline Parallelism

Data parallel replicates the model on each GPU, shards the batch, and all-reduces gradients. Simple, but model must fit on one GPU and communication is O(params) per step.

Tensor (model) parallel shards the model itself (e.g., split attention heads). Communication is inside the forward pass (all-gather after each matmul), more frequent and latency-sensitive — needs NVLink. Megatron-LM uses 2-D sharding.

Pipeline parallel assigns layers to GPUs and streams micro-batches through. Bubble overhead $(P-1)/M$ for M micro-batches; interleaved schedules (GPipe, PipeDream) shrink the bubble. In practice, large LLM training uses 3-D parallelism: data × tensor × pipeline, plus ZeRO sharding of optimizer states.

Remark 6.1. ZeRO (Zero Redundancy Optimizer) shards optimizer states, gradients, and optionally parameters across data-parallel ranks, trading extra all-gather for $N\times$ memory saving — the reason 175B models train on 1024 GPUs without 10 TB per GPU.

6.5 Multi-Node, MPI, and Practical Tips

Single-node 8 GPUs share NVLink; multi-node adds InfiniBand (NDR 400 Gb/s ≈50 GB/s per NIC, HDR 200 Gb/s). NCCL detects IB and uses `IB verbs` plus GPUDirect RDMA (GPU memory directly to NIC, bypassing CPU). Without GPUDirect, data copies GPU→CPU→NIC, halving BW.

Launch: one rank per GPU via `mpirun -np 8 -map-by ppr:8:node` or `torchrun`. Each rank sets `cudaSetDevice(rank%8)` and creates its NCCL communicator with a shared `ncclUniqueId` broadcast via MPI. Environment: `NCCL_DEBUG=INFO` prints the chosen algorithm (ring/tree/collnet) and bus BW; `NCCL_IB_DISABLE=1` forces PCIe for debugging.

Example 6.1 (Sizing the job)**.** BERT-large (340M params, fp16 → 0.68 GB). All-reduce 0.68 GB over NVLink 900 GB/s: ring cost $2\times0.68\times7/8/900 \approx 1.3$ ms. Compute per step ∼80 ms on 8 GPUs. $E \approx 80/(80+1.3) = 98\%$ — near linear. GPT-3 175B (350 GB fp16): all-reduce 350 GB even over 900 GB/s is 0.68 s vs. 2 s compute — $E \approx 75\%$ without ZeRO. ZeRO-3 shards params, so each all-reduce is smaller but you pay an all-gather per layer.

Debugging checklist: (1) `nvidia-smi topo -m` shows NVLink vs. PCIe; (2) `nccl-tests/all_reduce_perf -b 1M -e 1G` benchmarks BW; (3) if BW is PCIe-like on an NVLink box, check `NCCL_P2P_DISABLE` not set; (4) NCCL timeout after 1800 s usually means one rank crashed — check `dmesg` for Xid; (5) for hangs, `NCCL_DEBUG=INFO NCCL_DEBUG_SUBSYS=INIT,COLL` pinpoints the stuck collective.

Definition 6.2 (3-D parallelism)**.** Combining data (shard batch), tensor (shard matmul), and pipeline (shard layers) parallelism simultaneously. Typical LLM recipe: tensor = 8 within node (NVLink), data = 128 across nodes (IB), pipeline = 8 across stages. The product $8\times128\times8 = 8192$ GPUs is chosen so each collective stays within its fast domain.

6.6 Collective Tuning and Debugging at Scale

NCCL's autotuner picks `Simple`, `LL`, or `LL128` protocol per message size: `LL` is low-latency for <64 KB, `Simple` is high-BW for large. You can force `NCCL_PROTO=Simple` to test, but let autotune win in production.

For multi-node, check fabric: `ibstat`, `nvidia-smi topo -m`, and `NCCL_DEBUG=INFO` should show `Channel 00/01 ... type IB`. If it shows `SOC` or `SHM` across nodes, IB is misconfigured and you are on TCP (∼1 GB/s, 50× slower).

Definition 6.3 (GPUDirect RDMA)**.** A path where the NIC reads GPU HBM directly via PCIe BAR,

bypassing host memory. Requires `nvidia-peermem` / `dmabuf` and IB `rverbs`. Without it, NCCL copies `GPU→Host→NIC`, doubling latency and halving BW.

At 10k-GPU scale, even NCCL's ring can cause network incast. Hierarchical collectives do reduce-scatter within node (NVLink), then across nodes (IB), then all-gather within node. NCCL `collnet` implements this when `NCCL_ALGO=CollNet` and switches support SHARP in-network reduction.

6.7 Exercises

Exercise 6.1. An 8-GPU DGX needs to all-reduce 2 GB of fp16 gradients. Estimate time over NVLink 900 GB/s vs. PCIe 64 GB/s using ring cost $2N(P-1)/P$/BW. What efficiency $E(8)$ if compute is 400 ms and comm is your estimate? How does NVSwitch change the BW term?

Exercise 6.2. Explain why ring all-reduce is bandwidth-optimal ($\approx 2N$ per GPU independent of P) but latency-suboptimal. For $N = 4$ KB and $P = 16$, why does NCCL prefer tree? Compute hop counts for both.

Exercise 6.3. Contrast data-parallel vs. tensor-parallel communication pattern: when does each happen, what size, and how does NVLink vs. InfiniBand affect the choice? Give one example where model parallel is unavoidable.

Exercise 6.4. Write pseudo-code for overlapping backward compute with all-reduce using two CUDA streams and an event per layer. Where would a missing `cudaStreamWaitEvent` cause a race? How do you verify overlap with Nsight Systems?

Exercise 6.5. Pipeline parallelism with $P = 8$, $M = 32$ micro-batches: compute bubble fraction for naive GPipe. How does interleaved 1F1B reduce it? Why does ZeRO-3 add an all-gather before forward and what does it save?

6.8 Chapter Notes

NCCL docs: `docs.nvidia.com/deeplearning/nccl`; Thakur et al. "Optimization of Collective Communication" (ring analysis); Rashidi et al. "Data Center Scale Multi-GPU" (topology); Shoeybi et al. Megatron-LM (tensor parallel); Rajbhandari et al. ZeRO (memory sharding); Narayanan et al. "Efficient Large-Scale Language Model Training on GPU Clusters" (3-D parallelism survey). For InfiniBand across nodes, see `NCCLTests` and `nccl-tests`.

Chapter 7

Profiling: Nsight, Occupancy, and Roofline

You cannot optimise what you cannot measure — profile first, hypothesise, then code.

From zero: measurement before magic. We treat the GPU as a black box with counters. Profiling reveals whether time is spent computing, moving data, or waiting idle. We start with a mental model (time breakdown), then tools that fill it with numbers: Nsight Systems for the big picture, Nsight Compute for the kernel, and the roofline for the verdict.

Learning Outcomes

After this chapter you will be able to:

(L1) run Nsight Systems and Compute to collect timelines and kernel metrics;

(L2) interpret occupancy limits from registers, shared memory, and threadblock size;

(L3) plot a roofline and classify kernels as memory- vs. compute-bound;

(L4) use NVTX ranges to correlate source regions with GPU activity;

(L5) design an iterative optimisation loop driven by profiler data.

7.1 The Profiling Mindset

Optimisation without data is superstition. The first step is a *time budget*: for an end-to-end run, what fraction is kernel compute, H2D/D2H copy, NCCL, and CPU overhead? If copies are 40%, kernel tuning is premature — hide copies with streams first.

Two complementary views: *system* (Nsight Systems) shows CPU+GPU+NCCL on a shared timeline, answering "what runs when and who waits?"; *kernel* (Nsight Compute) shows one kernel's microarchitecture counters, answering "why is this kernel slow?" Use Systems first to find the bottleneck kernel, then Compute to fix it.

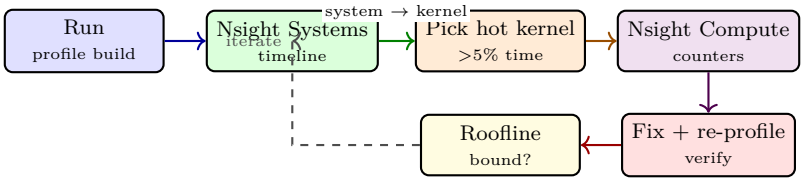


Figure 7.1: Profile-driven loop. Always start system-wide, drill to kernel, hypothesise a bound, fix one thing, and re-measure — one variable at a time.

7.2 Nsight Systems: The Timeline Truth

Run: `nsys profile -o report ./app` then `nsys stats report.nsys-rep` or open in GUI. You see rows: CPU threads, CUDA API, kernels, memory copies, NCCL, OS runtime. Gaps are idle — usually synchronisation or insufficient work.

Annotate code with *NVTX*: `nvtxRangePushA("forward")/Pop` or `NVTX3`. Ranges appear as coloured spans on the timeline, linking source functions to GPU activity without guessing from kernel names. Mark data-loader, forward, backward, optimizer, and comm phases distinctly.

What to look for: (i) kernel gaps → CPU bottleneck or sync, (ii) D2H copies blocking → need pinned memory + async, (iii) low GPU utilisation → bigger batch or CUDA graphs, (iv) NCCL overlapping or not.

Listing 7.1: NVTX annotation: mark phases so Nsight Systems shows them as colored spans.

```
#include <nvtx3/nvToolsExt.h>
void train_step(){
  nvtxRangePushA("data-load");
  load_batch(); // H2D copy here
  nvtxRangePop();
  nvtxRangePushA("forward");
  launch_forward(); // kernels
  nvtxRangePop();
  nvtxRangePushA("backward+allreduce");
  launch_backward(); ncclAllReduce(...);
  nvtxRangePop();
}
// nsys profile -t cuda,nvtx,osrt -o out ./train
// nsys stats --report gputrace out.nsys-rep
```

Typical Nsight Systems view: top row shows H2D copies (green), then Kernel Forward (orange), Kernel Backward (violet), then NCCL (red). A visible *gap* of idle time between Forward and Backward indicates a synchronisation or CPU bottleneck. NVTX spans (data-load, forward+backward, comm) colour the timeline and let you attribute each GPU segment to source code without guessing kernel names. If gaps exceed 10% of total, fix overlap and launch overhead before tuning kernels.

7.3 Nsight Compute: Inside One Kernel

Run: `ncu -set full -o k ./app` or `ncu -metrics sm__throughput.avg.pct, dram__throughput.avg.pct`. The report gives: achieved occupancy, warp stall reasons, memory throughput, instruction mix, and source-correlated hotspots.

Key counters include `sm__warps_active` (occupancy-like), `dram__throughput`, `l1tex__throughput`, and `smsp__sass_average_branch_targets` (divergence). Occupancy limits appear as `launch__occupancy_limit_registers`, `shared`, `blocks`. If `Memory` > `Compute` and `dram` ∼90%, you are memory-bound — tiling or fusion helps, not more math. If `Compute` ∼90% but `Tensor` pipe idle, you are not using Tensor Cores.

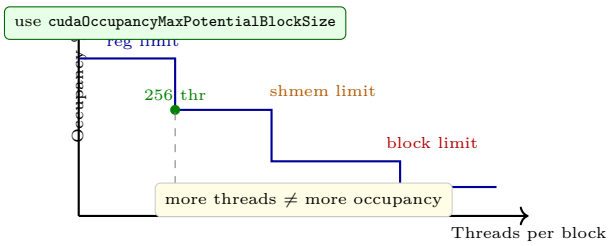
Stall reasons guide fixes: `short scoreboard` → data dependency, `lg throttle` → too many outstanding loads, `mio throttle` → instruction fetch, `not selected` → low occupancy.

7.4 Occupancy: How Many Warps Hide Latency?

Occupancy = active warps / max warps per SM. Higher hides latency better, but not always faster — a kernel with 50% occupancy but using more registers per thread (hence more ILP) can beat 100% with spills.

Definition 7.1 (Occupancy limiters)**.** For an SM with $W_{\max}$ warps, R registers, S shared memory: occupancy is limited by $\min(W_{\max},\ \lfloor R/(r\cdot 32)\rfloor,\ \lfloor S/s\rfloor,\ \lfloor B_{\max}/b\rfloor)$ where r = regs/thread, s = shmem/block, b = threads/block. The smallest wins.

Example: Hopper SM: $W_{\max}$=64 warps, R=65536 regs. Kernel uses r=64 regs/thread (2048 regs/warp), s=48 KB/block, b=256 threads (8 warps/block), S=228 KB. Register limit: 65536/2048=32 warps; shmem limit: 228/48=4 blocks → 32 warps; thread limit: 2048/256=8 blocks → 64 warps. So occupancy = 32/64=50%, limited by registers + shmem.



Query at runtime: `cudaOccupancyMaxPotentialBlockSize` returns the block size that maximises occupancy for the kernel's resource usage. Compile with `-Xptxas -v` to see regs/shmem per kernel. Reducing registers via `__launch_bounds__` or `-maxrregcount` can raise occupancy at cost of spills — measure, do not assume.

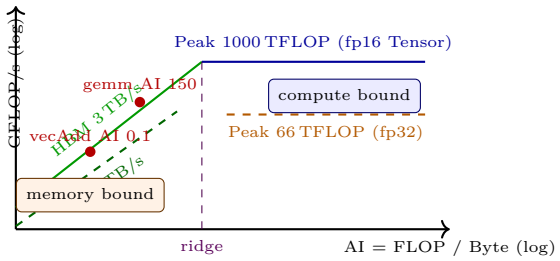
Listing 7.2: Occupancy API: query optimal block size and launch with it.

```
#include <cuda_runtime.h>
__global__ void myKernel(float* x, int n){ /* ... */ }
int minGrid, bestBlock;
cudaOccupancyMaxPotentialBlockSize(&minGrid, &bestBlock, myKernel, 0, 0);
printf("best block %d minGrid %d\n", bestBlock, minGrid);
int n=1<<20; int grid=(n+bestBlock-1)/bestBlock;
myKernel<<<grid, bestBlock>>>(dX, n);
// compare vs hand-picked 256: profile both with ncu
```

7.5 Roofline: The Verdict Plot

Arithmetic intensity AI = FLOP/byte from DRAM. Attainable = min(Peak, BW × AI). On log-log axes, the slanted part is memory-bound, the flat part compute-bound, the knee is the ridge $\text{AI}_{\text{ridge}} = \text{Peak/BW}$.

Place your kernel: `ncu` gives FLOP count and DRAM bytes; compute AI and mark the point. If below the roof, you have headroom; if on the roof, you are optimal for that bound. Optimisations move the point: tiling moves right (higher AI), vectorisation/Tensor Cores move up (higher effective peak), caching moves to a higher BW roof (L2/HBM).



Example: vector add does 1 FLOP per 12 B (2 reads +1 write) → AI≈0.08, firmly memory-bound; gemm N=4096 does $2N^3/(3N^2 \cdot 2\text{B}) \approx 1365$ in fp16 → compute-bound. Roofline tells you not to waste time unrolling vecAdd's math.

7.6 Putting It Together: Iterative Optimisation

A common workflow for a Transformer inference backend:

1. `nsys` shows end-to-end: 12% H2D, 68% compute, 20% gaps. Fix gaps first: capture CUDA graphs, use pinned memory, remove `cudaDeviceSynchronize` inside loop. Re-run: gaps 4%. 2. `nsys` now shows one kernel `softmax` at 22% of compute but low occupancy. `ncu` says `dram__throughput 88%`, `sm__throughput 12%` — memory-bound, and `short scoreboard` stalls dominate. Fusion: fuse softmax + dropout + residual into one kernel. AI rises from 0.5 to 3.5, time drops 2×. 3. New bottleneck: `gemm` at 45%. `ncu` says `sm__throughput 92%` but `tensor_throughput 18%` — not using Tensor Cores because layout is NCHW. Switch to NHWC or use `cublasLt` with `CUBLASLT_ORDER_COL32`. Tensor utilisation jumps to 85%, time 3×. 4. Roofline check: after fusion and Tensor, remaining kernels sit near the HBM roof — you are done unless you change the algorithm (e.g., flash attention).

Rule: fix system bottlenecks first (overlap, graphs), then memory bottlenecks (fusion, tiling), then compute bottlenecks (Tensor Cores, ___launch_bounds___). Each step needs a before/after `ncu` diff: one metric moves, not all.

Remark 7.1. Occupancy calculator spreadsheet (NVIDIA) vs. runtime API: the spreadsheet is planning, the API is truth. The spreadsheet assumes worst-case registers; compiler may allocate fewer. Always confirm with `-Xptxas -v` and `ncu launch__registers_per_thread`.

7.7 Common Pitfalls and Checklist

`nsys` overhead is ∼5% in default mode, ∼20% with `-trace=cuda,nvtx,osrt,cudnn,cublas`. Use `-sample=none` for production runs and `-capture-range=cudaProfilerApi` to profile only the region between `cudaProfilerStart/Stop`.

`ncu` replays the kernel many times (once per metric set). A `-set full` can replay >10 times, making a 1 ms kernel take 5 s. For quick iteration, use `-set roofline` or `-metrics dram__throughput,sm__throughput` (single replay). Always `cudaDeviceSynchronize()` before and after the profiled region to avoid mixing.

Checklist before claiming a speedup: (1) pinned host memory, (2) warmup, (3) fixed clocks (`nvidia-smi -lgc`), (4) same power limit, (5) same input size, (6) re-profile after each single change.

7.8 Exercises

Exercise 7.1. You run `nsys` and see 35% GPU idle gaps between kernels, each gap ∼15 μs. Name two CPU-side causes and two GPU-side fixes (CUDA graphs, batching, streams, pinned memory). How would NVTX help attribute the gaps?

Exercise 7.2. A kernel uses 48 KB shared memory, 64 regs/thread, 256 threads/block. On an SM with 228 KB shmem, 65536 regs, 64 max warps, compute occupancy and the limiter. What block size or `__launch_bounds__` change would raise it, and what risk?

Exercise 7.3. `ncu` reports 40 GFLOP and 20 GB DRAM read+write for a kernel on Hopper (peak 1000 TFLOP fp16, BW 3 TB/s). Compute AI and attainable performance. Is it memory or compute bound? What single optimisation would move it most?

Exercise 7.4. Explain why Nsight Systems and Nsight Compute are complementary, not substitutes. Give a scenario where Systems says "kernel is 2% of time" but Compute says "kernel is 90% DRAM bound" — what do you optimise?

Exercise 7.5. Sketch a roofline with HBM, L2, fp32 and Tensor peaks. Place three kernels: (a) batched small gemm N=32, (b) large gemm N=8192 fp16, (c) softmax. Mark which ceiling each hits and propose one fix per kernel to ride a higher roof.

7.9 Chapter Notes

Nsight Systems `docs.nvidia.com/nsight-systems`, Nsight Compute `docs.nvidia.com/nsight-compute` and the "Ncu Metrics" guide; Volkov "Better Performance at Lower Occupancy" (GTC); Williams et al. "Roofline: An Insightful Visual Performance Model" (CACM 2009); Yang et al. "Instruction Roofline" for GPU. For occupancy, see CUDA Programming Guide App. G and `cudaOccupancyMaxPotentialBlockSize` docs. Use `nsys stats -report gputrace` for CI-friendly summaries.

Chapter 8

Applications: AI, HPC, Graphics, and the Future

From training GPT to tracing light — same silicon, wildly different stories.

From zero: application first, GPU second. We start from what users want — faster learning, faster science, prettier pictures — and show how the same GPU primitives (massive threads, Tensor Cores, ray tracing cores, NVLink) serve each. Every domain maps to a pattern taught earlier: GEMM, stencil, or tree traversal.

Learning Outcomes

After this chapter you will be able to:

(L1) map AI training/inference to GEMM-heavy kernels and explain Tensor Core speedups;

(L2) describe two HPC GPU patterns (CFD stencil, molecular dynamics) and their scaling;

(L3) explain GPU graphics pipeline stages and how ray tracing/DLSS use GPU cores;

(L4) compare Hopper, Blackwell, and Grace-Hopper architectures and their new primitives;

(L5) discuss future trends: power wall, wafer-scale, photonics, and sustainability.

8.1 AI: Training and Inference Dominate

Modern AI is matrix multiplication. A Transformer layer is: QK^T (batched GEMM), softmax, AV (batched GEMM), two feed-forward GEMMs, plus LayerNorm and residual. For GPT-3 scale, $> 98\%$ of FLOPs are GEMMs — exactly what Tensor Cores accelerate 8–16× over fp32 CUDA cores via `fp16/bf16` and `fp8`.

Training is throughput-bound (large batches, data-parallel across hundreds of GPUs with NCCL). Inference is latency-bound (small batch, often single token) — techniques like KV-cache, flash attention (fusion to cut

HBM traffic), and speculative decoding matter more than raw FLOPs. *CUDA Graphs* capture a whole inference step as one launch, removing kernel-launch overhead that dominates at batch 1.

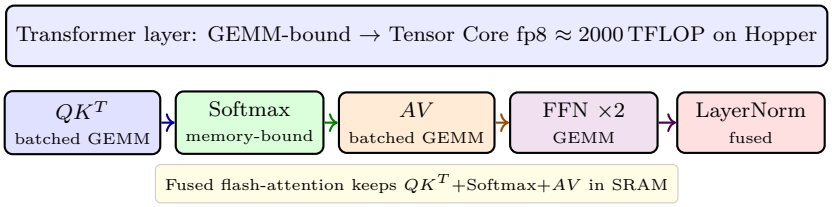


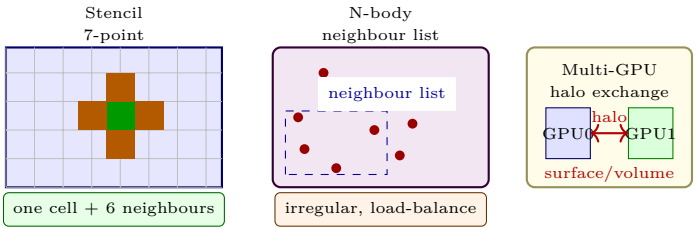
Figure 8.1: Transformer layer breakdown. GEMMs dominate FLOPs; softmax and LayerNorm dominate memory traffic unless fused. Flash attention fuses the middle three into one kernel.

Framework view: PyTorch `torch.compile` and TensorRT capture graphs, fuse pointwise ops, and call CUTLASS/cuBLASLt kernels autotuned for your shapes — the library story from Ch. 5 delivering 2–5× over eager mode.

8.2 HPC: Science at Scale

CFD (computational fluid dynamics) solves Navier-Stokes on a 3-D grid: each step is a *stencil* (each cell reads its 6 neighbours). Arithmetic intensity is low (∼1–2 FLOP/B) — memory-bound. GPUs win via high HBM BW and shared-memory tiling; multi-GPU needs halo exchange (NCCL or MPI) whose cost grows with surface-to-volume ratio.

Molecular dynamics (e.g., GROMACS, LAMMPS) computes N-body forces: $O(N^2)$ naively, $O(N)$ with Particle-Mesh Ewald or Barnes-Hut. Short-range forces are neighbour-list kernels (irregular), long-range is FFT (communication-bound). GPUs accelerate both, but load balancing across GPUs is hard when particles cluster.



Both domains use mixed precision: fp32 for accumulation, fp16/bf16 for force computation — same numerical trick as AI.

8.3 Graphics: Real-Time Light

The classic pipeline: vertex shading → rasterization → fragment shading → blending. Rasterization is embarrassingly parallel (one thread per triangle or per pixel). Modern GPUs add *RT Cores* for ray tracing (BVH tree traversal + ray-triangle intersection) and *Tensor Cores* for DLSS (Deep Learning Super Sampling): render at 540p, upscale to 4K with a neural net, gaining 2–3× fps.

Ray tracing is incoherent (random memory access into BVH) and divergent (rays scatter). Techniques from Ch. 4 matter: sorting rays by direction, compacting hit/miss paths, and using shared memory for BVH nodes.

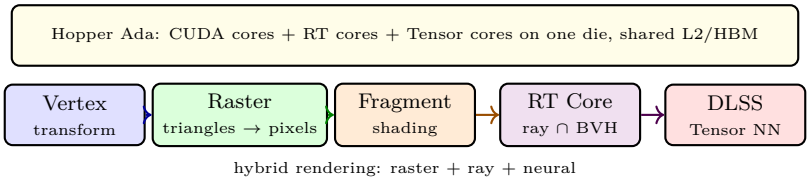


Figure 8.2: Modern graphics pipeline. Raster, ray tracing, and neural upscaling share the same GPU, each favouring a different core type but all memory-intensive.

8.4 Hopper, Blackwell, Grace-Hopper, and Beyond

Hopper (H100) added: 4th-gen Tensor Core with `fp8` and Transformer Engine, `wgmma` (warp-group matrix multiply-accumulate) that lets a group of 4 warps cooperatively drive Tensor Cores, thread-block clusters for cross-SM shared memory, and DPX instructions for dynamic programming. HBM3 3 TB/s and NVLink 4.0 900 GB/s remove many bottlenecks from Ch. 6.

Blackwell (B100/B200) doubles HBM to 192 GB, adds 2nd-gen Transformer Engine with microscaling, NVLink 5.0 1.8 TB/s, and a decompress engine (helpful for LLM KV-cache). The chip is actually two dies on an interposer — a chiplet lesson from SC3050.

Grace-Hopper pairs a Grace ARM CPU and Hopper GPU with 900 GB/s NVLink-C2C coherent link and unified memory — pages migrate on demand, not explicit `cudaMemcpy`. The programming shift is from "two memories" to "one address space with affinity hints."

Generational trend: V100 (2017, 900 GB/s) → A100 (2020, 2 TB/s) → H100 (2023, 3 TB/s) → B200 (2024, 8 TB/s) → GH200 (CPU+GPU). Each generation roughly doubles HBM bandwidth, halves precision (fp16 → fp8 → fp4), and doubles NVLink (300 → 1800 GB/s). New primitives appear each time: `wgmma`, TMA async copy, DPX, and thread-block clusters.

8.5 Future: Power, Photonics, and Sustainability

Scaling is now power-limited, not transistor-limited. A DGX H100 draws 10 kW; an exascale cluster draws 20 MW. Future gains come from domain specialisation (like Hopper's Transformer Engine), tighter packaging (3-D stacking, chiplets), and moving compute near data (Grace's unified memory is a step toward near-memory).

Wafer-scale (Cerebras) and photonics (light-based interconnect) attack the same roofline from different angles: one removes chip boundaries, the other removes copper limits. Both must prove programmability and yield before displacing GPUs. The durable lesson is the roofline and Amdahl: any future machine wins by raising the right roof for your workload's AI and shrinking the serial fraction.

8.6 Sustainability, Confidential Computing, and Your Next Step

Power is now a first-class metric alongside time. Reporting `performance/Watt` and `performance/CO_2` is becoming standard: embodied carbon (manufacturing) vs. operational carbon (running). A chiplet that improves yield by 30% cuts embodied carbon even if its BW is slightly lower — a trade architects now model explicitly.

NVIDIA's *confidential computing* (Hopper+) encrypts HBM and attests the GPU, so data stays encrypted even from the cloud provider — critical for private LLM inference. The trend is *sovereign* and *edge* GPUs: Grace-Hopper at rack scale for training, Jetson Orin at 15 W for robotics, both running the same CUDA code. Your code from Ch. 2–4 scales across them unchanged; only the library heuristic and roofline change.

Example 8.1 (What to learn next)**.** After this book: (1) pick one domain (AI, CFD, or graphics) and profile a real workload end-to-end with `nsys` + `ncu`; (2) re-implement its hot kernel once with cuBLASLt/CUTLASS and once with a custom kernel, compare roofline points; (3) scale it to 2–4 GPUs with NCCL and measure $E(P)$. That loop is the job of a GPU performance engineer — and now you have the toolkit.

Remark 8.1. The 0→100 journey in this book mirrors GPU history: start serial (CPU), discover data parallelism (SIMT), hit memory walls (coalescing, tiling), learn to hide latency (occupancy, streams), borrow libraries (cuBLAS), scale out (NCCL), and finally measure ruthlessly (Nsight, roofline). The future is the same loop at larger scale.

Listing 8.1: Hopper wgmma: warp-group matmul using CUTLASS 3.x / cute. Four warps cooperate.

```
// CUTLASS 3 cute: warp-group MMA (128x128x32 tile, fp8)
using TileMNK = Shape<_128,_128,_32>;
using Atom = SM90_64x128x32_F32F8F8F32_TN; // Tensor Core atom
// TMA async copy: global -> shared, then wgmma
cute::copy(Copy_Atom<SM90_TMA_LOAD>{}, gA, sA); // async
cute::copy(Copy_Atom<SM90_TMA_LOAD>{}, gB, sB);
warpgroup_arrive();
cute::gemm(Atom{}, sA, sB, sC); // wgmma.m64n128k32
warpgroup_commit_batch();
warpgroup_wait<0>();
// sC now holds 128x128 fp32 accumulator, epilogue writes to global
```

Listing 8.2: CUDA Graphs for low-latency inference: capture once, replay cheaply.

```
import torch
model = torch.compile(model)  # or torch.cuda.make_graphed_callables
# Capture
g = torch.cuda.CUDAGraph()
with torch.cuda.graph(g):
    out = model(inp)  # kernels recorded, not just computed
# Replay: one launch, not N
for _ in range(1000):
    g.replay()        # ~5 us vs 50 us eager
    torch.cuda.synchronize()
```

8.7 Putting It All Together: End-to-End Example

Consider training a 7B LLM on 32 Hopper GPUs: data-parallel 8 × tensor-parallel 4. Each step: forward GEMMs (cuBLASLt fp8), flash attention (fused, Ch. 7), backward (same), NCCL all-reduce 0.5 GB grads over NVLink+IB (∼2 ms), optimizer (fused Adam, Thrust-like). Nsight shows 85% GPU util, roofline shows GEMMs on Tensor roof, attention on HBM roof — healthy. Scaling from 8 to 32 GPUs: $E = 0.88$; the 12% loss is IB all-reduce plus pipeline bubble — exactly the Ch. 6 model.

8.8 Exercises

Exercise 8.1. A Transformer layer with d=4096, seq 2048, batch 4 does four 4096×4096 GEMMs per token. Estimate FLOPs per layer in fp16 and time on H100 (1000 TFLOP fp8 vs. 66 TFLOP fp32). Why does flash attention improve time even though FLOPs are unchanged?

Exercise 8.2. CFD stencil on 1024^3 grid, 7-point, 100 iterations: estimate bytes moved and AI. Is it memory or compute bound on H100 (3 TB/s, 1000 TFLOP)? How does halo exchange over NVLink vs. PCIe affect multi-GPU strong scaling at 8 GPUs?

Exercise 8.3. Explain why ray tracing is incoherent and divergent compared with rasterization. Name two optimisations (ray sorting, compaction) and map each to a Ch. 4 pattern (partition, stream compaction). How do RT cores help?

Exercise 8.4. Compare Hopper `wgmma` vs. Ampere `mma.sync`: who participates, what tile size, and why does warp-group cooperation increase Tensor Core utilisation? Relate to CUTLASS `Atom` choice.

Exercise 8.5. A Grace-Hopper superchip has unified memory with 900 GB/s C2C. Contrast explicit `cudaMemcpy` vs. page migration for a 100 GB dataset touched once by CPU then repeatedly by GPU. When does unified memory hurt? Propose an affinity hint policy.

8.9 Chapter Notes

Hopper whitepaper `resources.nvidia.com/en-us-tensor-core` and CUTLASS 3.x docs (cute/wgmma/TMA); FlashAttention (Dao et al. 2022); GROMACS/LAMMPS GPU papers; NVIDIA Ada/Ray tracing Gems; Grace-Hopper architecture (Choquette & Gandhi, Hot Chips 2023); Blackwell announcement (GTC 2024). For systems trends, see Leiserson et al. “There’s Plenty of Room at the Top” (Science 2020) and Horowitz “Computing’s Energy Problem” (ISSCC 2014).